

移动应用开发实验报告

封面信息

项目	内容
实验名称	微信小程序开发
实验编号	d20301035104、d20301009204
学号	2023210603
姓名	李俊杰
班级	软件232
组别	组别1
日期	2026-04-23
项目名称	智能健康运动记录平台开发与整合
技术栈	微信小程序 (JavaScript)

实验目的

- 掌握微信小程序开发流程
- 学会WXML/WXSS/JS开发
- 理解页面路由与数据传递
- 掌握本地存储使用
- 体验AI辅助开发

实验环境

工具	版本	用途
微信开发者工具	最新版	小程序开发与调试
Node.js	18+	可选，用于npm依赖管理
AI辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 页面流程图

首页(通知列表) → 详情页(通知详情)
↓
本地存储(已读状态)

1.2 数据模型

```
// 通知数据结构
{
  id: 1,
  title: "关于期末考试安排的通知",
  content: "详细内容...",
  author: "教务处",
  time: "2024-01-01",
  isRead: false
}
```

2. 创建小程序项目

2.1 新建项目

- ☒ 打开微信开发者工具
- ☒ 新建小程序项目
- ☒ 填写AppID（如无则使用测试号）
- ☒ 选择JavaScript模板

3. 通知列表页面开发

3.1 WXML布局

```
<!-- pages/index/index.wxml -->
<view class="container">
  <view class="news-list">
    <view
      class="news-item {{item.isRead ? 'read' : ''}}"
      wx:for="{{newsList}}"
      wx:key="id"
      bindtap="goToDetail"
      data-id="{{item.id}}"
    >
      <view class="title">{{item.title}}</view>
      <view class="info">
        <text>{{item.author}}</text>
        <text>{{item.time}}</text>
      </view>
    </view>
  </view>
</view>
```

3.2 WXSS样式

```
/* pages/index/index.wxss */
.news-item {
  padding: 16px;
  border-bottom: 1px solid #eee;
  background: #fff;
}
```

```

.news-item.read {
  opacity: 0.6;
}
.title {
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 8px;
}
.info {
  font-size: 12px;
  color: #999;
  display: flex;
  justify-content: space-between;
}

```

3.3 JS逻辑

```

// pages/index/index.js
Page({
  data: {
    newsList: []
  },

  onLoad() {
    this.loadNews();
  },

  onPullDownRefresh() {
    this.loadNews().then(() => {
      wx.stopPullDownRefresh();
    });
  },

  loadNews() {
    // 模拟数据
    const newsList = [
      { id: 1, title: "关于期末考试安排的通知", content: "详细内容...", author: "教务处", time: "2024-01-01", isRead: false },
      { id: 2, title: "图书馆开放时间调整", content: "详细内容...", author: "图书馆", time: "2024-01-02", isRead: false },
      { id: 3, title: "校园网络维护通知", content: "详细内容...", author: "信息中心", time: "2024-01-03", isRead: false }
    ];

    // 读取本地存储的已读状态
    const readIds = wx.getStorageSync('readIds') || [];
    newsList.forEach(news => {
      news.isRead = readIds.includes(news.id);
    });

    this.setData({ newsList });
  },

  gotoDetail(e) {
    const id = e.currentTarget.dataset.id;

```

```
wx.navigateTo({
  url: `/pages/detail/detail?id=${id}`
});
}
```

3.4 配置文件

```
// pages/index/index.json
{
  "navigationBarTitleText": "智慧校园通知",
  "enablePullDownRefresh": true,
  "usingComponents": {}
}
```

4. 详情页开发

4.1 WXML布局

```
<!-- pages/detail/detail.wxml -->
<view class="container">
  <view class="title">{{news.title}}</view>
  <view class="info">
    <text>发布部门: {{news.author}}</text>
    <text>发布时间: {{news.time}}</text>
  </view>
  <view class="content">{{news.content}}</view>
</view>
```

4.2 JS逻辑

```
// pages/detail/detail.js
Page({
  data: {
    news: {}
  },

  onLoad(options) {
    const id = parseInt(options.id);
    this.loadDetail(id);
  },

  loadDetail(id) {
    // 模拟数据
    const allNews = [
      { id: 1, title: "关于期末考试安排的通知", content: "本次期末考试安排如下...",
        author: "教务处", time: "2024-01-01" },
      { id: 2, title: "图书馆开放时间调整", content: "图书馆开放时间调整...", author:
        "图书馆", time: "2024-01-02" },
      { id: 3, title: "校园网络维护通知", content: "网络维护通知...", author: "信息中
        心", time: "2024-01-03" }
    ];
```

```
const news = allNews.find(n => n.id === id);
this.setData({ news });

// 标记为已读
this.markAsRead(id);
},

markAsRead(id) {
  let readIds = wx.getStorageSync('readIds') || [];
  if (!readIds.includes(id)) {
    readIds.push(id);
    wx.setStorageSync('readIds', readIds);
  }
}
});
```

5. 本地存储功能

5.1 已读状态存储

```
// 保存已读ID
wx.setStorageSync('readIds', [1, 2, 3]);

// 读取已读ID
const readIds = wx.getStorageSync('readIds');

// 检查是否已读
const isRead = readIds.includes(newsId);
```

6. 测试与发布

6.1 真机预览

- ☒ 编译项目
- ☒ 扫描二维码预览
- ☒ 测试各项功能

6.2 体验版发布

- ☒ 点击上传
- ☒ 填写版本信息
- ☒ 获取体验版二维码

架构与设计

架构图

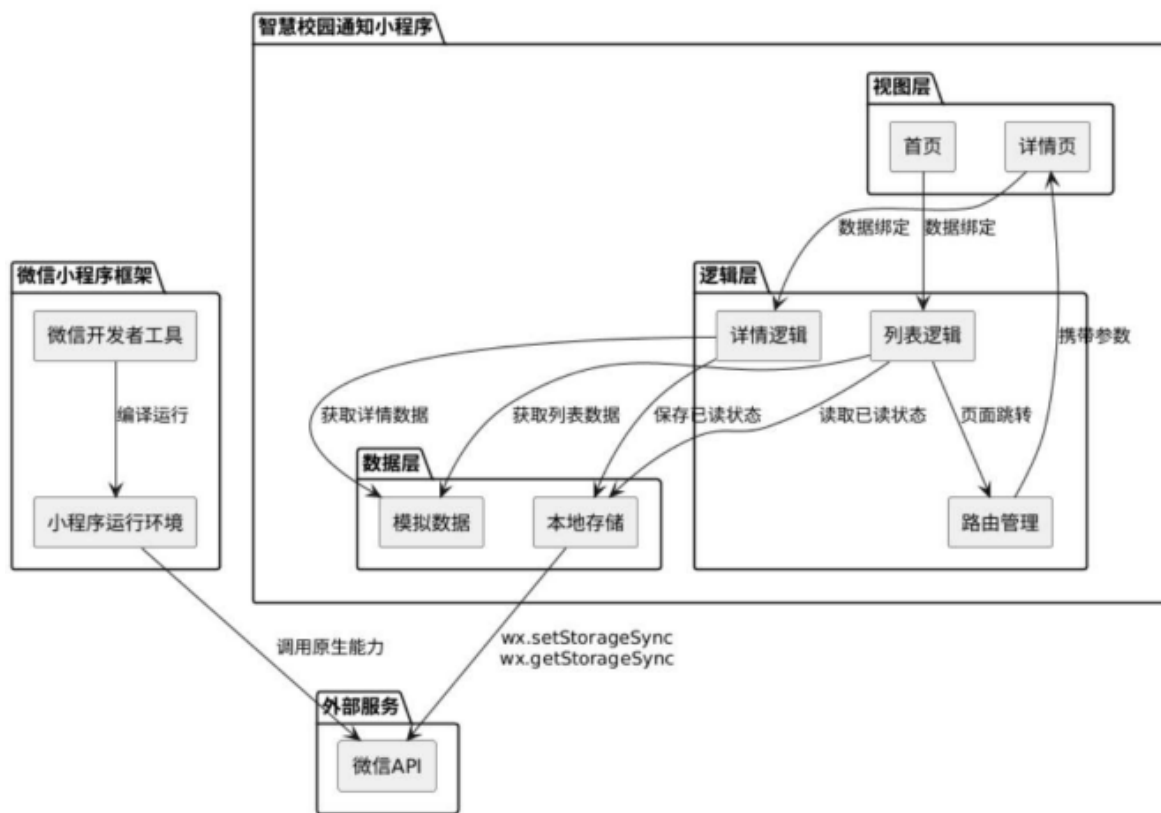


图1. 架构图

架构说明：

- **视图层**：包含首页（通知列表）和详情页两个页面组件，负责界面展示和用户交互
- **逻辑层**：处理页面交互逻辑、路由跳转和数据管理，是小程序的核心处理层
- **数据层**：管理本地存储和模拟数据，实现数据的持久化和获取
- 采用微信小程序标准的MVVM架构，数据通过本地存储实现持久化，页面间通过路由传递参数

类图

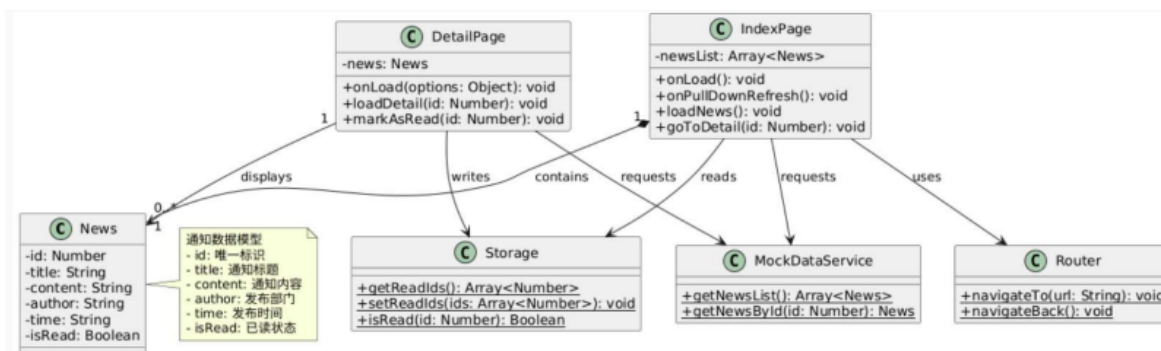


图2. 类图

类图说明：

- **数据模型类**：News类是通知数据实体，包含唯一标识、标题、内容、发布部门、发布时间和已读状态等属性
- **页面类**：IndexPage负责首页管理，包括通知列表展示、下拉刷新和页面跳转功能；DetailPage负责详情页管理，包括通知详情展示和已读标记功能
- **服务类**：Storage封装本地存储的读写操作；Router处理页面跳转；MockDataService提供模拟通知数据

时序图

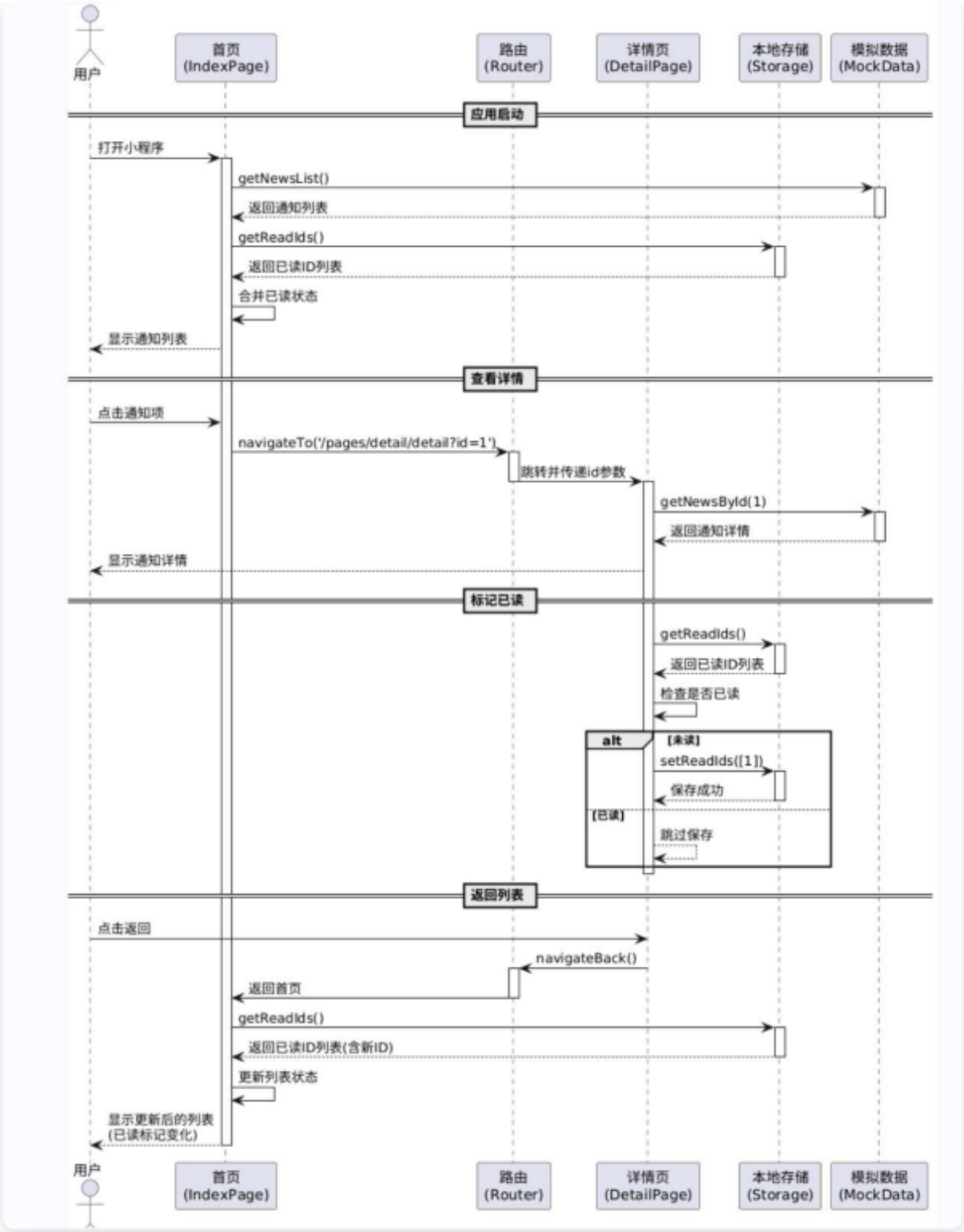


图3. 时序图

时序图说明：

1. 用户打开小程序后，首页从数据服务获取通知列表
2. 同时从本地存储读取已读状态，合并数据后显示通知列表
3. 用户点击某条通知后，通过路由跳转到详情页
4. 详情页获取通知详情并展示给用户
5. 详情页检查该通知是否已读，若未读则将ID保存到本地存储
6. 用户返回首页后，页面重新读取已读状态并更新列表显示

实验结果

验证结果

功能	状态	备注
通知列表展示	[x] 成功	正常显示通知列表
详情页跳转	[x] 成功	点击通知可跳转到详情页
已读状态存储	[x] 成功	本地存储已读状态
下拉刷新	[x] 成功	支持下拉刷新功能
真机预览	[x] 成功	小程序可在真机上正常运行

截图记录

图4. 通知列表页面



图5. 通知详情页面



图6. 已读状态效果



问题与解决

问题描述	解决方案	解决结果
本地存储数据丢失	使用wx.setStorageSync和wx.getStorageSync进行数据持久化	已解决
页面跳转参数传递	使用url参数传递ID	已解决
已读状态更新	在详情页标记已读并更新本地存储	已解决

实验总结

实验收获

- 掌握了微信小程序的开发流程，从项目创建到真机预览的完整链路
- 学会了WXML模板语法和WXSS样式编写
- 理解了小程序的页面路由机制和参数传递方式
- 掌握了本地存储API的使用，实现数据持久化
- 体验了AI辅助开发的流程，提升了开发效率

技术要点

- 微信小程序开发：使用微信开发者工具创建和管理项目
- 页面开发：使用WXML、WXSS和JavaScript进行页面开发
- 路由管理：使用wx.navigateTo进行页面跳转和参数传递
- 本地存储：使用wx.setStorageSync和wx.getStorageSync实现数据持久化
- 真机测试：通过微信开发者工具进行真机预览和测试

考核指标

考核项	评分标准	得分
功能完整性	通知列表、详情页、本地存储功能完整	30分
代码质量	代码结构清晰，注释完整	20分
页面设计	界面美观，用户体验良好	15分
本地存储	已读状态存储正常	15分
真机预览	小程序可在真机上正常运行	10分
实验报告	报告结构完整，内容详实	10分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名：_____

日期：_____